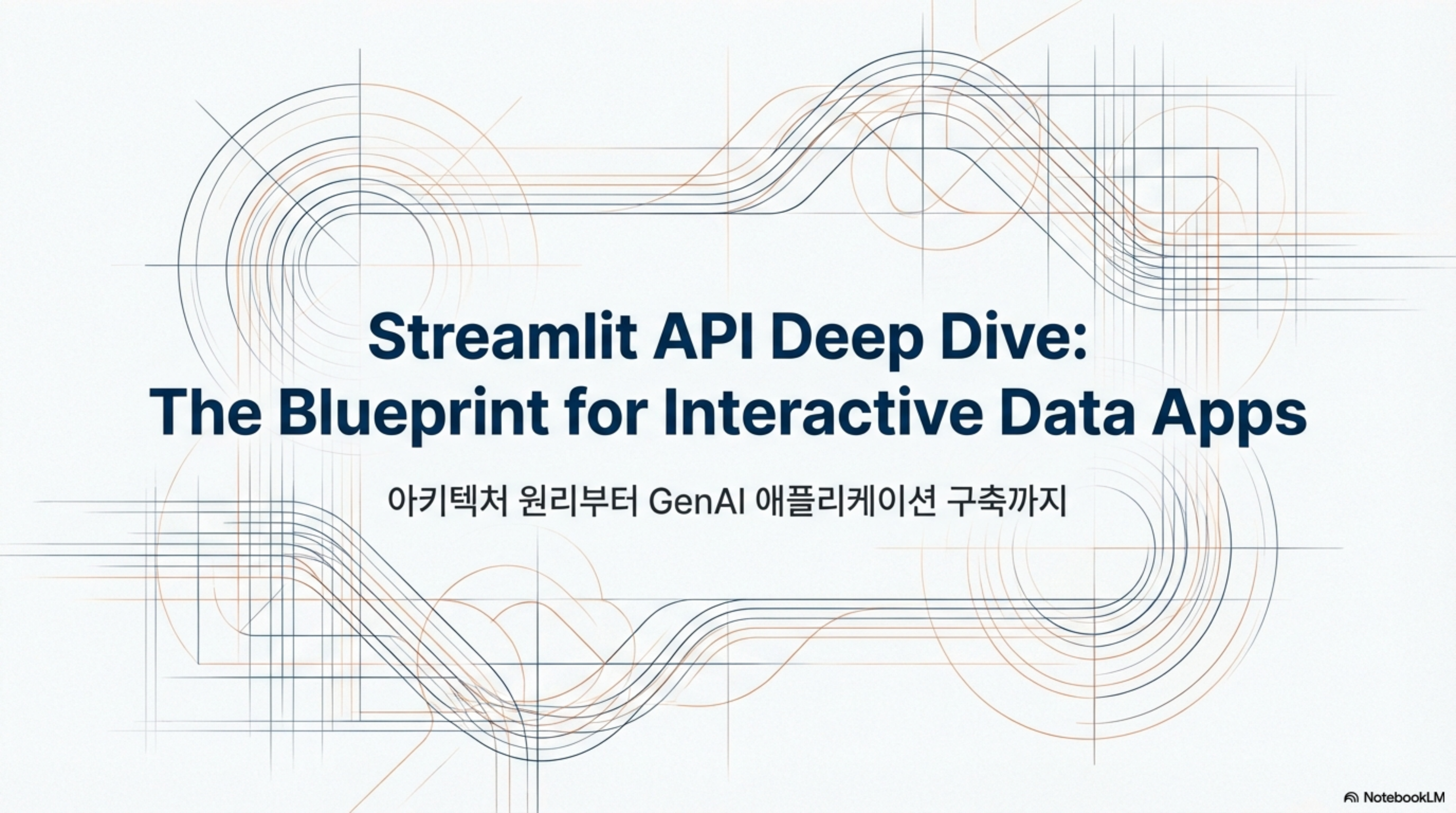




Streamlit API Deep Dive: The Blueprint for Interactive Data Apps

아키텍처 원리부터 GenAI 애플리케이션 구축까지

목표: Python 스크립트를 웹 앱으로, 즉시.

Streamlit은 데이터 과학자와 머신러닝 엔지니어가 웹 개발 지식 없이 순수 Python 스크립트만으로 대화형 웹 앱을 구축하도록 설계된 프레임워크입니다.

핵심 가치는 복잡한 웹 개발 과정(HTML, CSS, JS, 콜백 관리)을 제거하고 데이터 분석 및 모델링 작업에 집중할 수 있게 하는 것입니다.



Pure Python. No Callbacks. No HTML/CSS/JS.

모든 것의 시작: 스크립트 전체 재실행 (Top-to-Bottom Rerun)

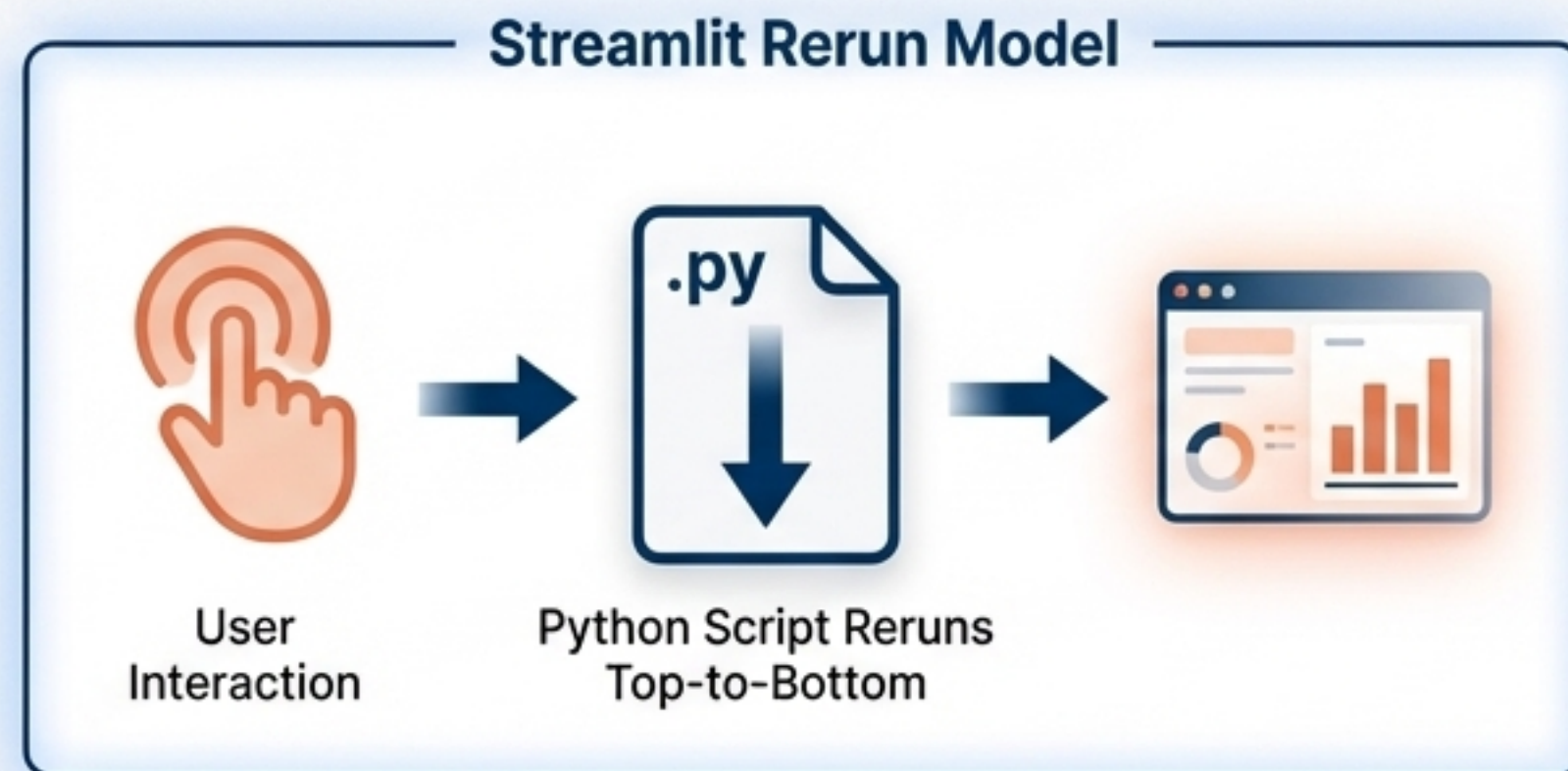
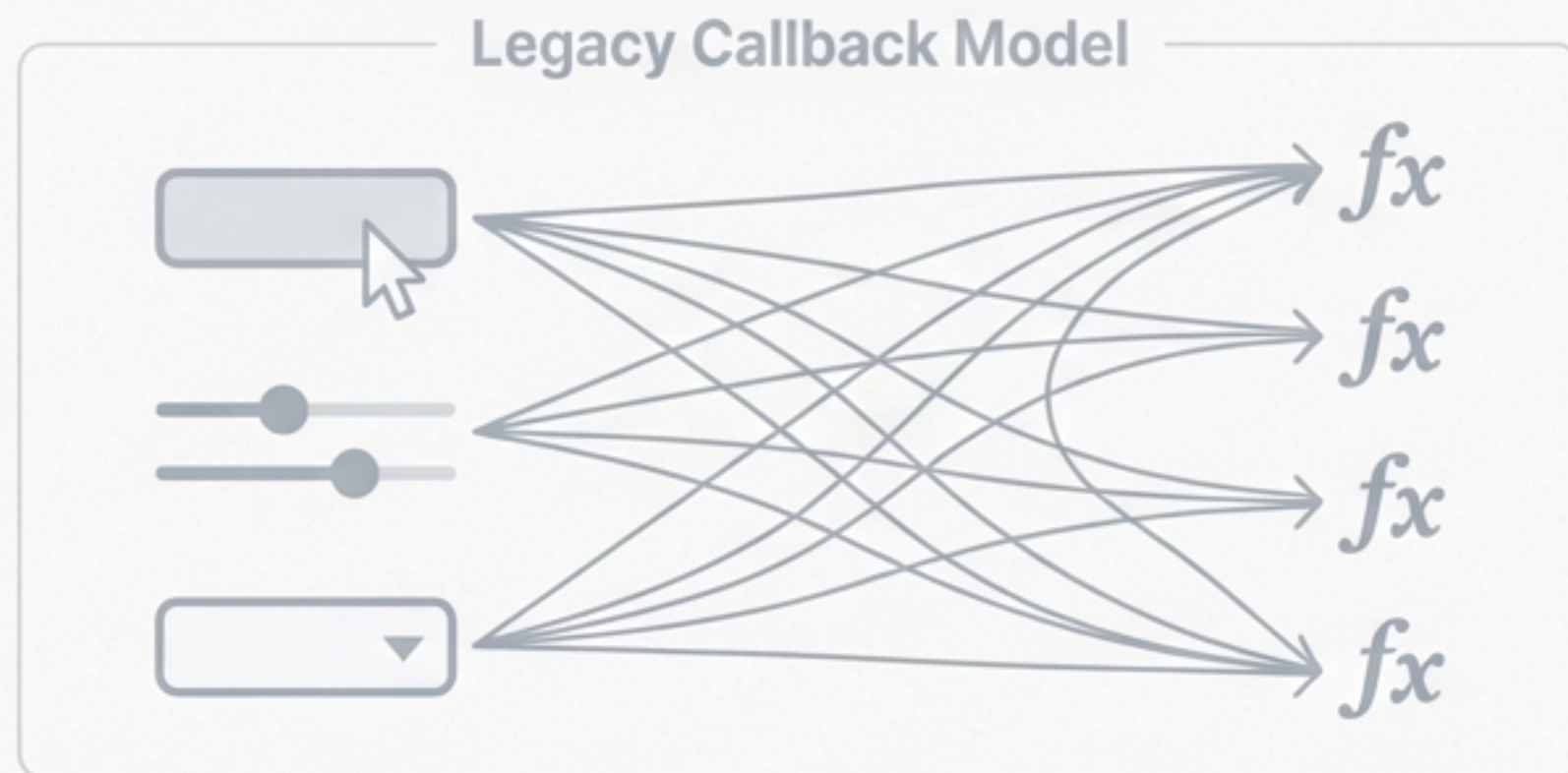
사용자가 위젯과 상호작용할 때마다 Streamlit은 전체 **Python** 스크립트를 최상단부터 최하단까지 다시 실행합니다.

이벤트 기반 프레임워크의 복잡한 콜백(Callback) 로직 대신, 선언적이고 반응형인 프로그래밍 모델을 채택했습니다.

중요: `on\_change`나 `on\_click` 같은 콜백은 스크립트의 나머지 부분이 재실행되기 직전에 실행되어, 상태 업데이트 완장합니다.

Architectural Benefits

- 선언적 UI 구조: 코드가 절차적 스크립트처럼 읽기 쉬워집니다.
- 반응형 프로그래밍 모델: 데이터 변경 시 의존하는 모든 로직이 자동으로 다시 계산됩니다.



브라우저와 서버의 대화법: 클라이언트-서버 아키텍처

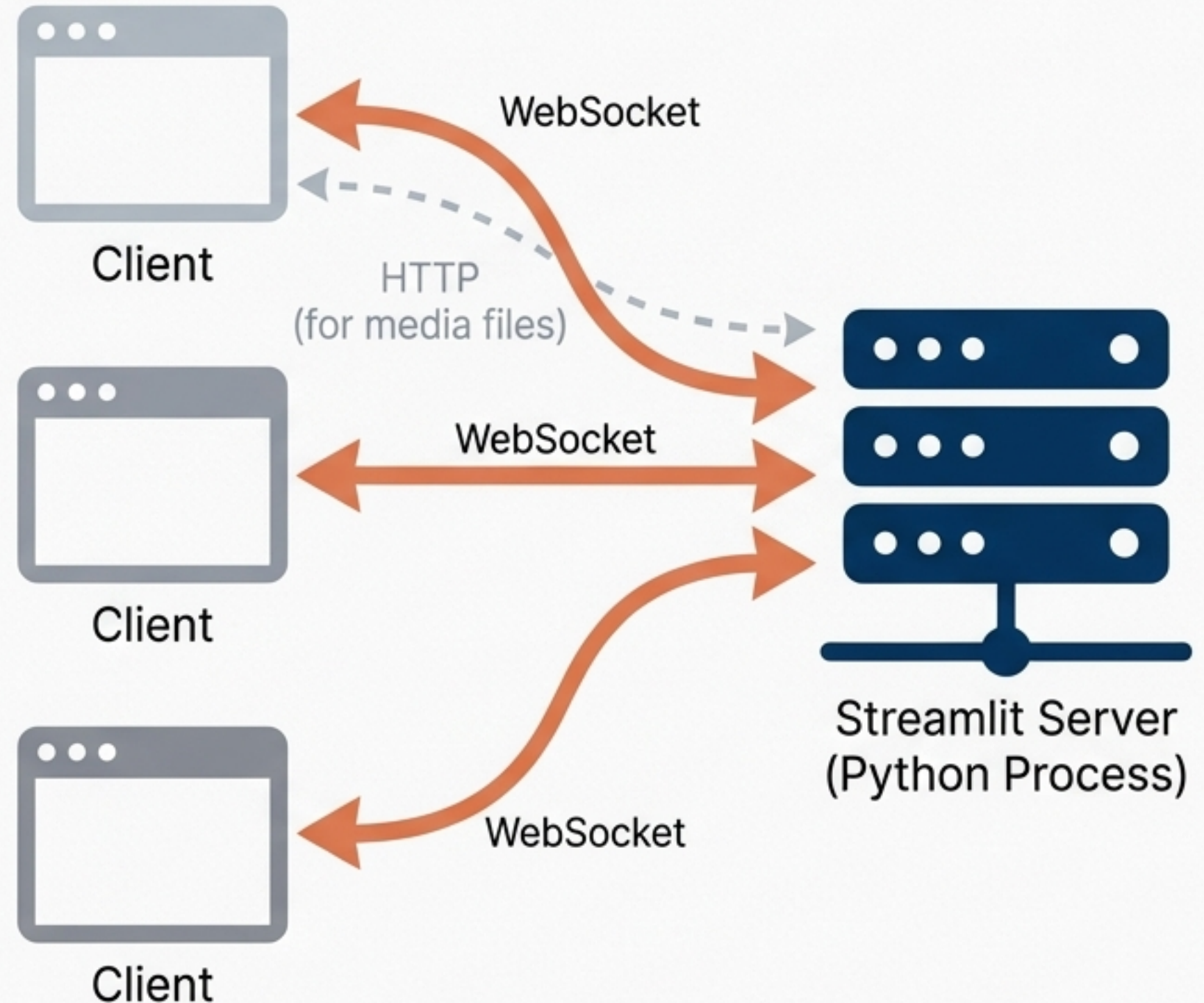
서버 (Python Backend): `streamlit run` 실행 시 시작되는 프로세스. 모든 계산과 로직 처리를 담당하는 '두뇌' 역할을 합니다.

클라이언트 (Browser): 사용자가 접속하는 브라우저.

통신: Tornado 기반의 WebSocket 연결을 통해 지속적인 양방향 통신을 유지합니다.

Important Technical Details:

- **세션 격리:** 각 브라우저 탭은 고유한 세션으로 관리되어 사용자 간 상호작용이 격리됩니다.
- **미디어 처리:** 이미지/오디오 파일은 WebSocket이 아닌 별도 HTTP 요청으로 로드됩니다. 로드 밸런서 환경에서는 **Session Affinity** 설정이 필수적일 수 있습니다.



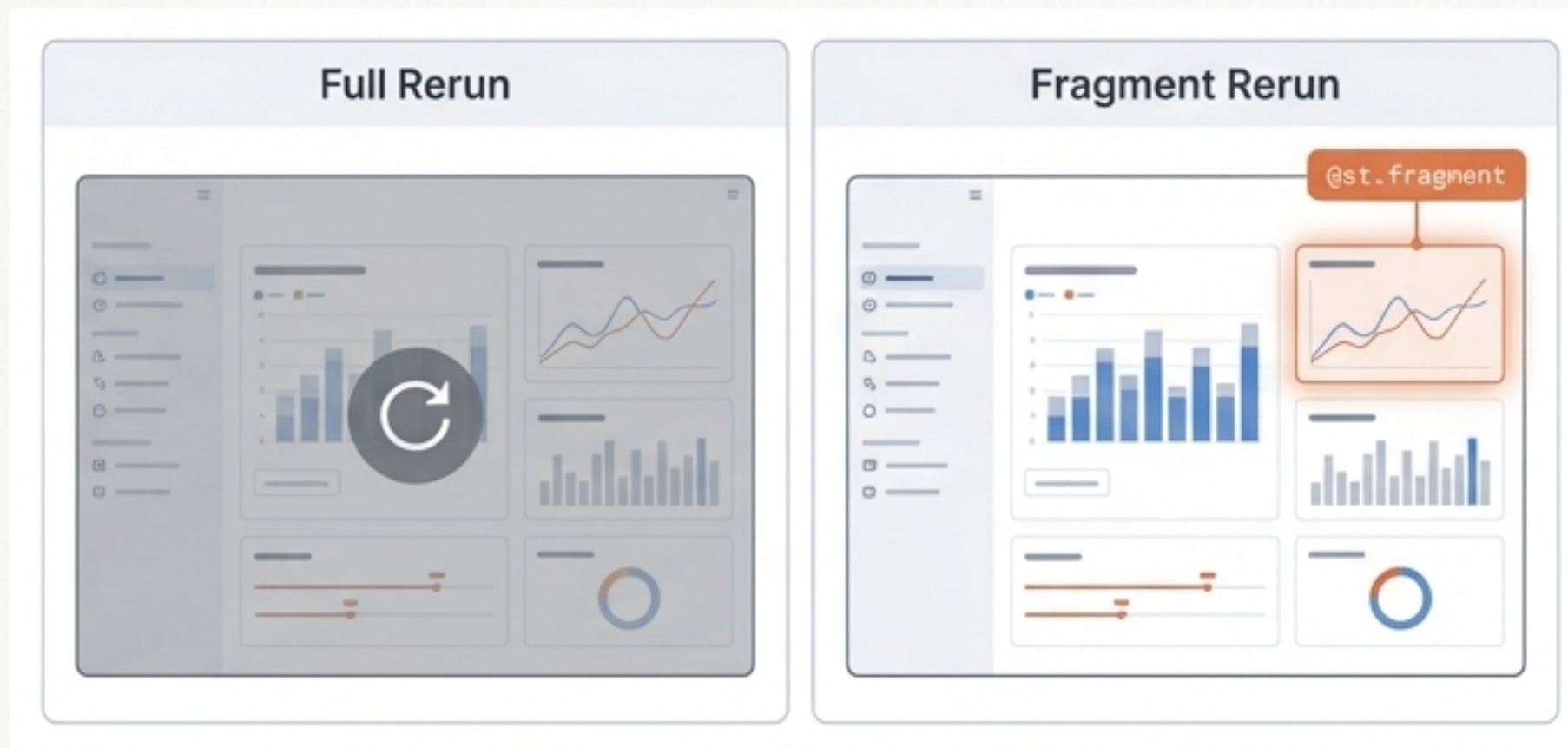
실행 흐름 제어하기: 정지, 재시작, 그리고 부분 업데이트

- Basic Control Flow

- `st.stop()`: 현재 지점에서 스크립트 실행을 즉시 중단합니다. (예: 필수 입력값이 없을 때)
- `st.rerun()`: 스크립트를 최상단부터 강제로 다시 실행합니다. (예: 상태 변경을 즉시 반영할 때)

Advanced Optimization: `st.fragment`

- **개념**: `@st.fragment` 데코레이터가 적용된 함수는 독립적인 실행 컨텍스트를 가집니다.
- **동작**: 프래그먼트 내부 위젯과 상호작용 시, **전체 앱이 아닌 해당 프래그먼트만 재실행됩니다.**
- **아키텍처적 이점**: 전체 페이지를 다시 그리지 않아 렌더링 성능이 비약적으로 향상됩니다. 차트 필터링이나 부분 데이터 업데이트에 이상적입니다.



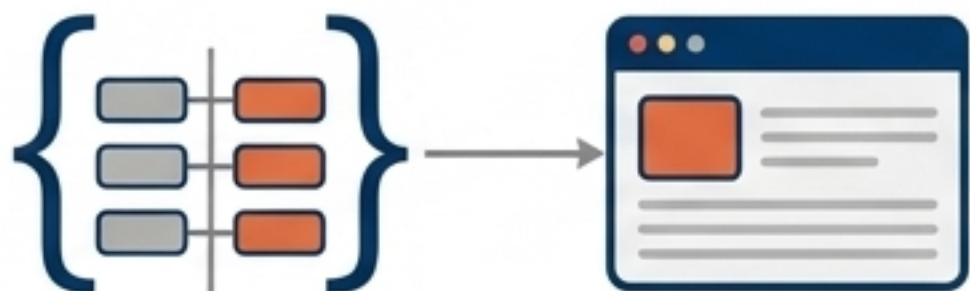
재실행을 넘어 데이터 유지하기: 상태 관리와 캐싱

Problem: 매 재실행마다 스크립트 내 변수는 초기화됩니다.

Solution 1: 세션 내 상태 유지 (`st.session\_state`)

용도: 사용자의 단일 세션 동안 데이터를 유지하기 위한 딕셔너리 형태의 키-값 저장소입니다.

핵심 기능: 위젯의 `key` 파라미터를 지정하면 `st.session\_state`와 값이 자동으로 동기화됩니다.



Solution 2: 고비용 연산 최적화 (Caching Decorators)

@st.cache_data: 직렬화 가능한 데이터 (예: 데이터프레임, API 호출 결과)를 저장합니다. 입력 인자가 동일하면 함수를 재실행하지 않고 캐시된 결과를 반환합니다.

@st.cache_resource: 직렬화 불가능한 객체 (예: DB 연결, ML 모델)를 저장합니다. 앱 전체에서 리소스를 한 번만 로드하고 공유합니다.



이 두 가지 개념을 마스터하는 것이 복잡하고 성능 좋은 앱을 만드는 핵심입니다.

정보 표현의 기술: 텍스트와 데이터

- The Magic Command: `st.write()`
 - 입력된 데이터 타입에 따라 최적의 렌더링 방식을 자동으로 결정하는 범용 함수입니다.
 - 예: DataFrame -> `st.dataframe()`, 문자열 -> `st.markdown()`, 함수 -> `st.help()`
- **Typography for Structure**
 - `st.title()`, `st.header()`, `st.subheader()`: 문서의 계층 구조 정의
 - `st.markdown()`: GitHub-flavored Markdown 지원
 - `st.code()`: 구문 강조 기능이 포함된 코드 블록 표시
 - `st.latex()`: 수학 수식 렌더링
- **Core Data Display**
 - `st.dataframe()`: 정렬, 검색, 스크롤이 가능한 **인터랙티브 테이블**
 - `st.metric()`: KPI를 강조 표시. `'delta'` 값으로 이전 대비 변화량을 시각적으로 보여줌.

앱 타이틀

This is **bold** text and *italic* text.

```
import streamlit as st
```

₩1.2B

▲ +5.4%

Column A	Column B	Column C
Data 1	123	True
Data 2	456	False
Data 3	338	False
Data 4	123	True

데이터를 넘어: UI에서 직접 데이터 편집하기

Introducing `st.data_editor`:

`st.dataframe`의 모든 기능을 포함하며, 사용자가 UI에서 직접 데이터를 수정, 추가, 삭제할 수 있습니다.

`num_rows="dynamic"` 파라미터로 행 추가/삭제 기능을 활성화할 수 있습니다.
반환값: 수정된 데이터는 원본 데이터 타입(예: Pandas DataFrame)으로 즉시 반환됩니다.

Unlocking Power with `st.column_config`:

각 컬럼의 표시 형식과 편집 방식을 정밀하게 제어합니다.

`st.column_config.NumberColumn`: 통화 기호, 최소/최대값 제한 설정.

`st.column_config.TextColumn`: 정규표현식(Regex)으로 입력 검증.

`st.column_config.LinkColumn`: 텍스트를 클릭 가능한 하이퍼링크로 변환.

`st.column_config.ProgressColumn`: 0-100 사이 값을 프로그레스 바로 시각화.

`st.column_config.ChartColumn`: 셀 내부에 스파크라인(Sparkline) 표시.

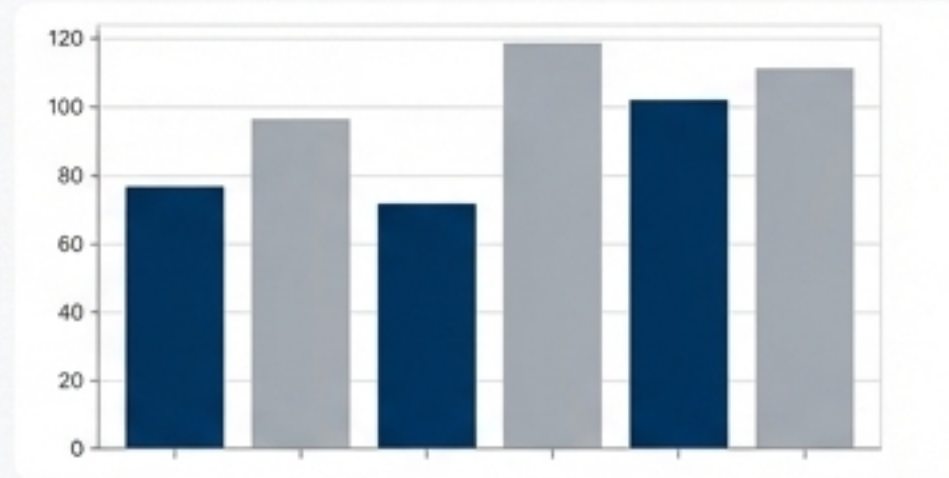
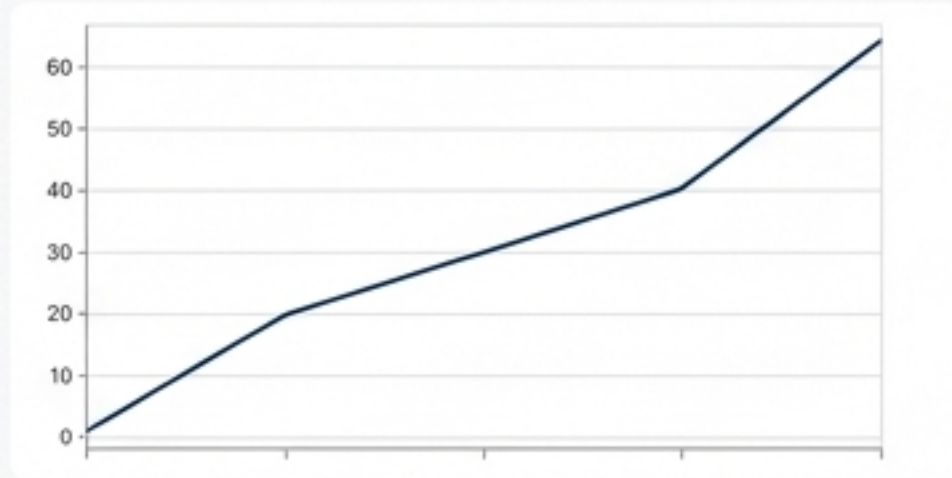
`st.column_config.ChartColumn`: 셀 내부에 스파크라인(Sparkline) 표시.

제품명	재고	판매량	달성률	링크
Laptop Pro	120 		75%	link-to-laptop.com
Smart Watch	85 		90%	link-to-watch.com
Wireless Earbuds	200 		40%	link-to-earbuds.com
Tablet	60 		100%	link-to-tablet.com

데이터 시각화: 내장 차트와 전문 라이브러리 통합

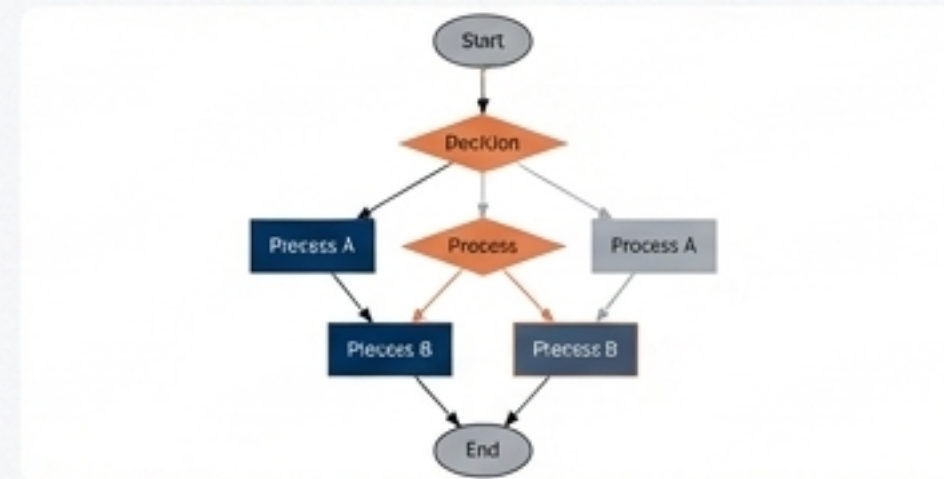
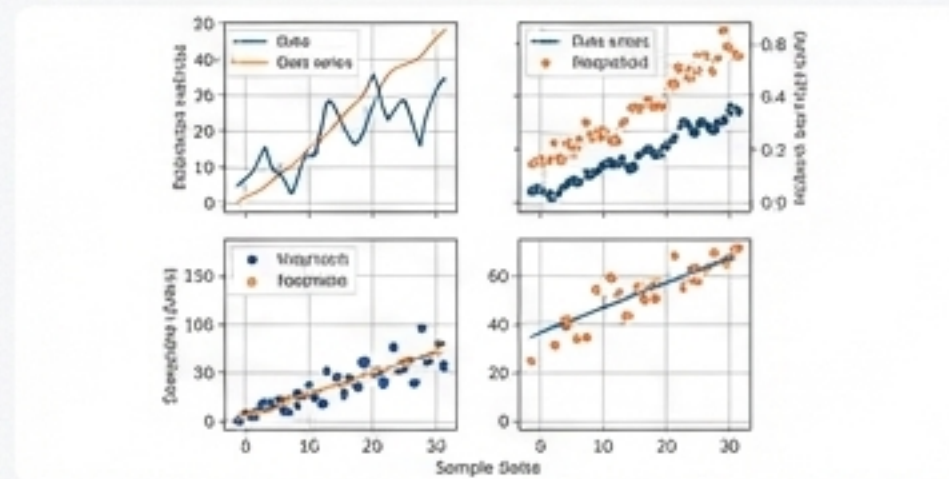
Native Charts for Quick Exploration (Altair 기반)

``st.line_chart()`, `st.bar_chart()`, `st.scatter_chart()`, `st.map()``



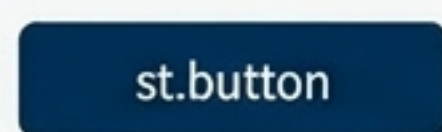
Advanced Integration with Python Libraries

Streamlit은 Python 생태계의 주요 시각화 라이브러리를 완벽하게 지원합니다.
``st.plotly_chart()`, `st.altair_chart()`, `st.pyplot()`, `st.graphviz_chart()``



사용자 상호작용의 핵심: 입력 위젯

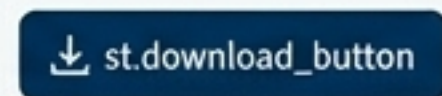
동작 트리거 (Action Triggers)



``st.button``:
알회성 동작 실행



``st.form()`` &
``st.form_submit_button``:
여러 입력값을 한 번에
묶어(batch) 제출.
불필요한 재실행 방지.

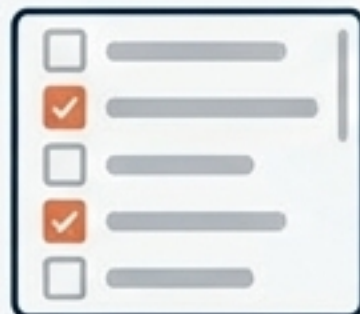


``st.download_button``:
데이터 다운로드 기능
제공

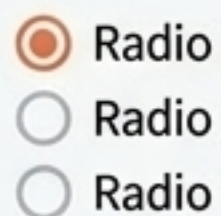
항목 선택 (Selection)



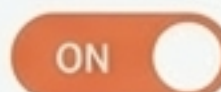
``st.selectbox``,
``st.radio``:
단일 항목 선택



``st.multiselect``:
다중 항목 선택

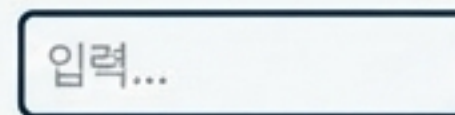


``st.radio``

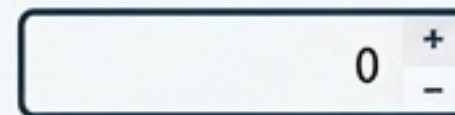


``st.toggle``:
On/Off 상태 전환

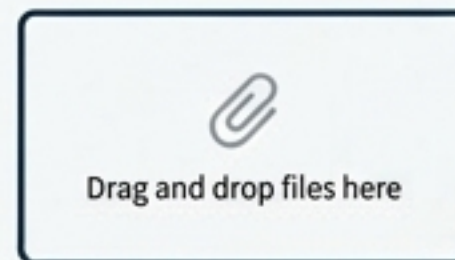
데이터 입력 (Data Entry)



``st.text_input``
(`type="password"` 지원),
``st.text_area``:
텍스트 입력



``st.number_input``,
``st.slider``:
숫자 및 범위 입력



``st.file_uploader``:
파일 업로드

모든 위젯은 ``key`` 파라미터를 통해 ``st.session_state``에 자신의 상태를 직접 연결할 수 있습니다.

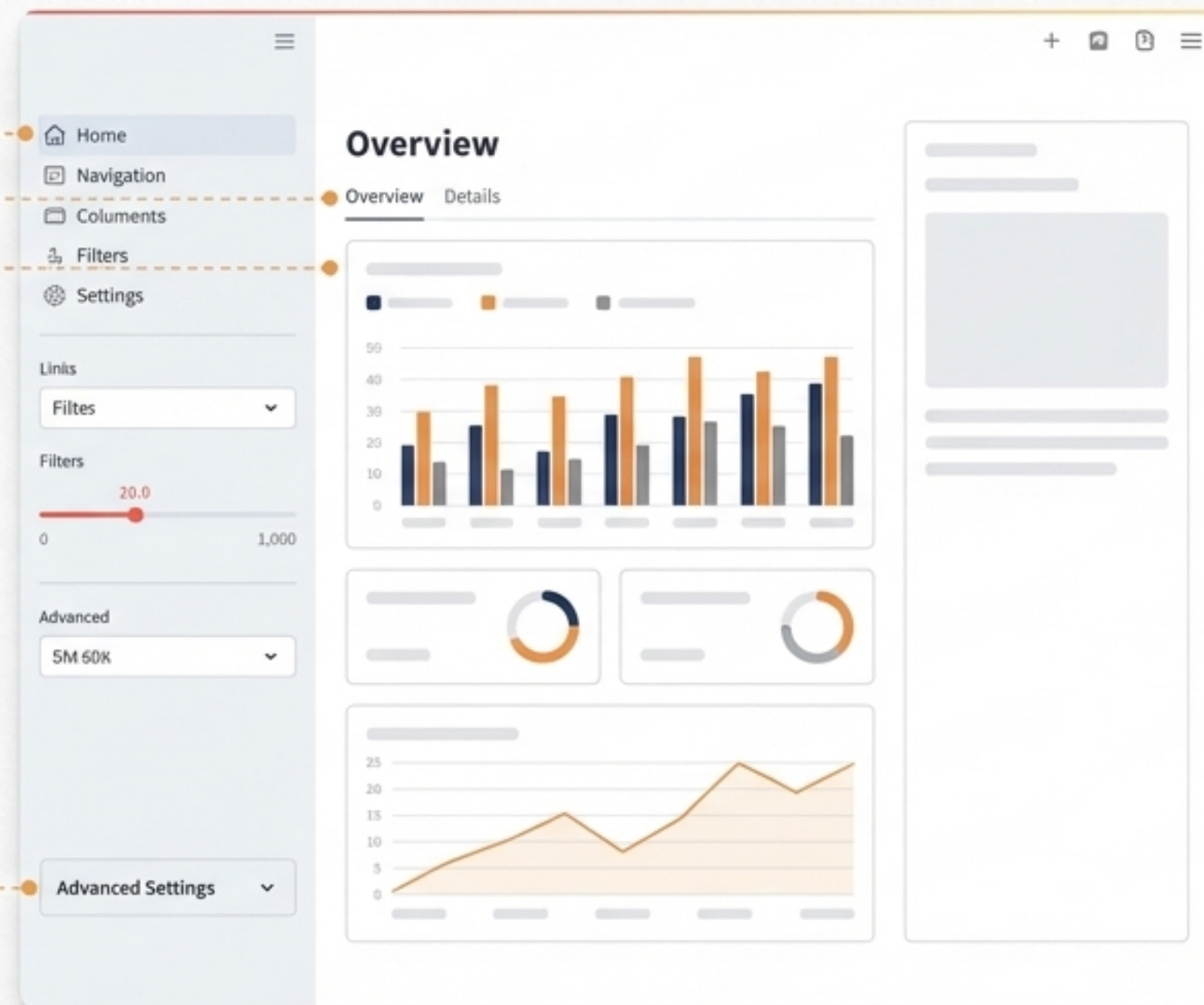
앱의 뼈대 세우기: 레이아웃과 컨테이너

Screen Layout Primitives

- **st.sidebar**: 화면 왼쪽에 고정된 사이드바. 전역 필터나 네비게이션에 이상적입니다.
- **st.columns()**: 화면을 여러 개의 수직 컬럼으로 분할합니다. (예: `st.columns([2, 1])`는 2:1 비율로 분할)
- **st.tabs()**: 콘텐츠를 주제별로 분리하는 탭 인터페이스를 생성합니다.

Content Containers

- **st.expander()**: 클릭 시 내용을 펼치거나 숨기는 접이식 컨테이너.
- **st.container()**: 여러 요소를 하나의 논리적 단위로 그룹화합니다.
- **st.empty()**: 나중에 내용을 덮어쓰거나 비울 수 있는 플레이스홀더. 실시간 업데이트 구현에 필수적입니다.



GenAI 시대의 표준: 대화형 AI 앱 구축

The Chat Elements Trio

`st.chat_message(name, avatar):`

"user" 또는 "assistant"의 채팅 메시지를 위한 컨테이너입니다.

- 특징: 내부에 텍스트뿐만 아니라 차트, 데이터프레임 등 모든 Streamlit 요소를 포함하여 **멀티모달(Multimodal) 인터페이스** 구현이 가능합니다.

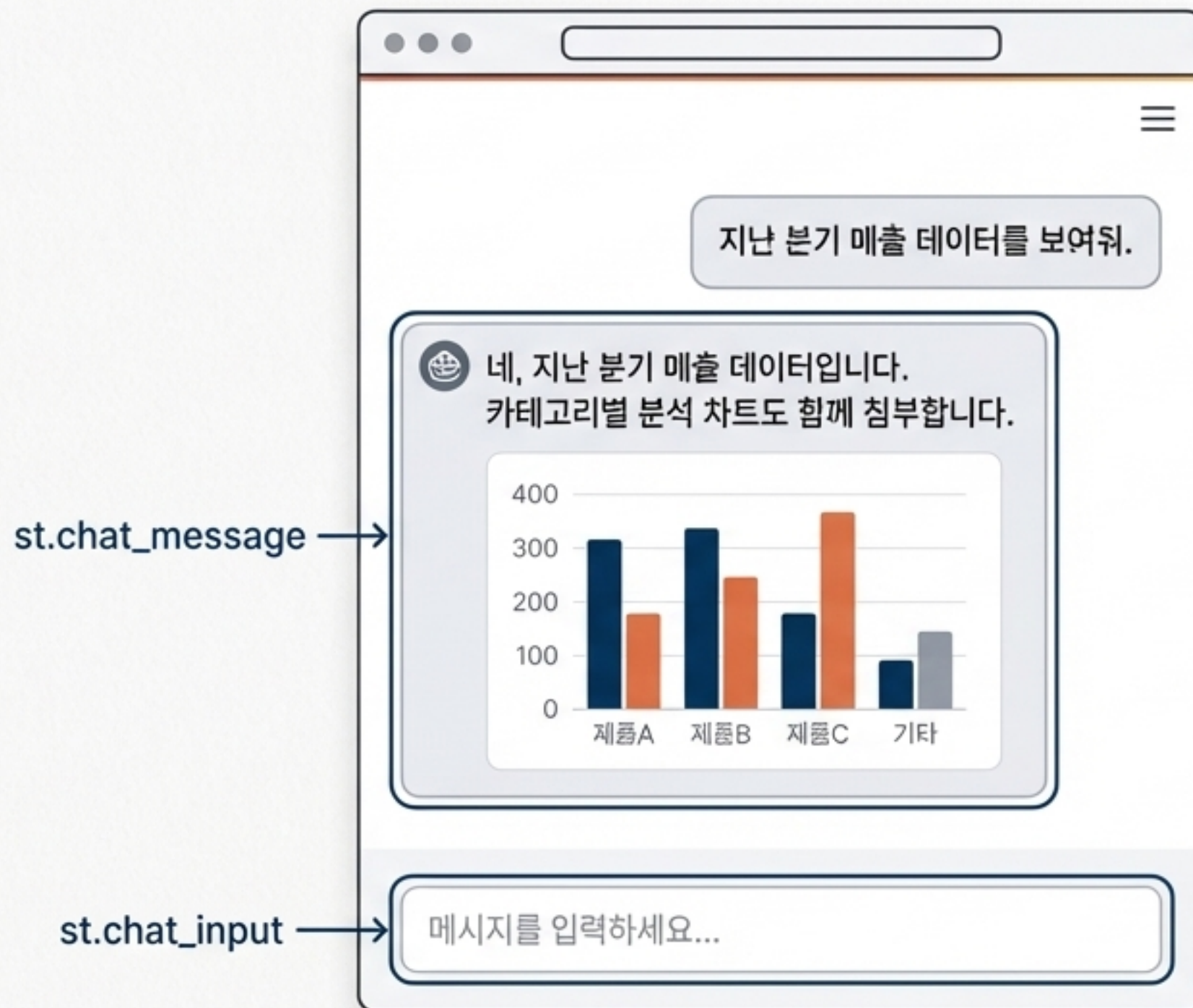
`st.chat_input():`

화면 하단에 고정되는 사용자 메시지 입력 위젯입니다.

`st.write_stream():`

제너레이터(Generator)를 입력받아 LLM 응답을 타자기 효과(Typewriter effect)로 실시간 출력합니다.

- OpenAI, LangChain 등 주요 라이브러리의 스트리밍 응답과 직접 호환됩니다.



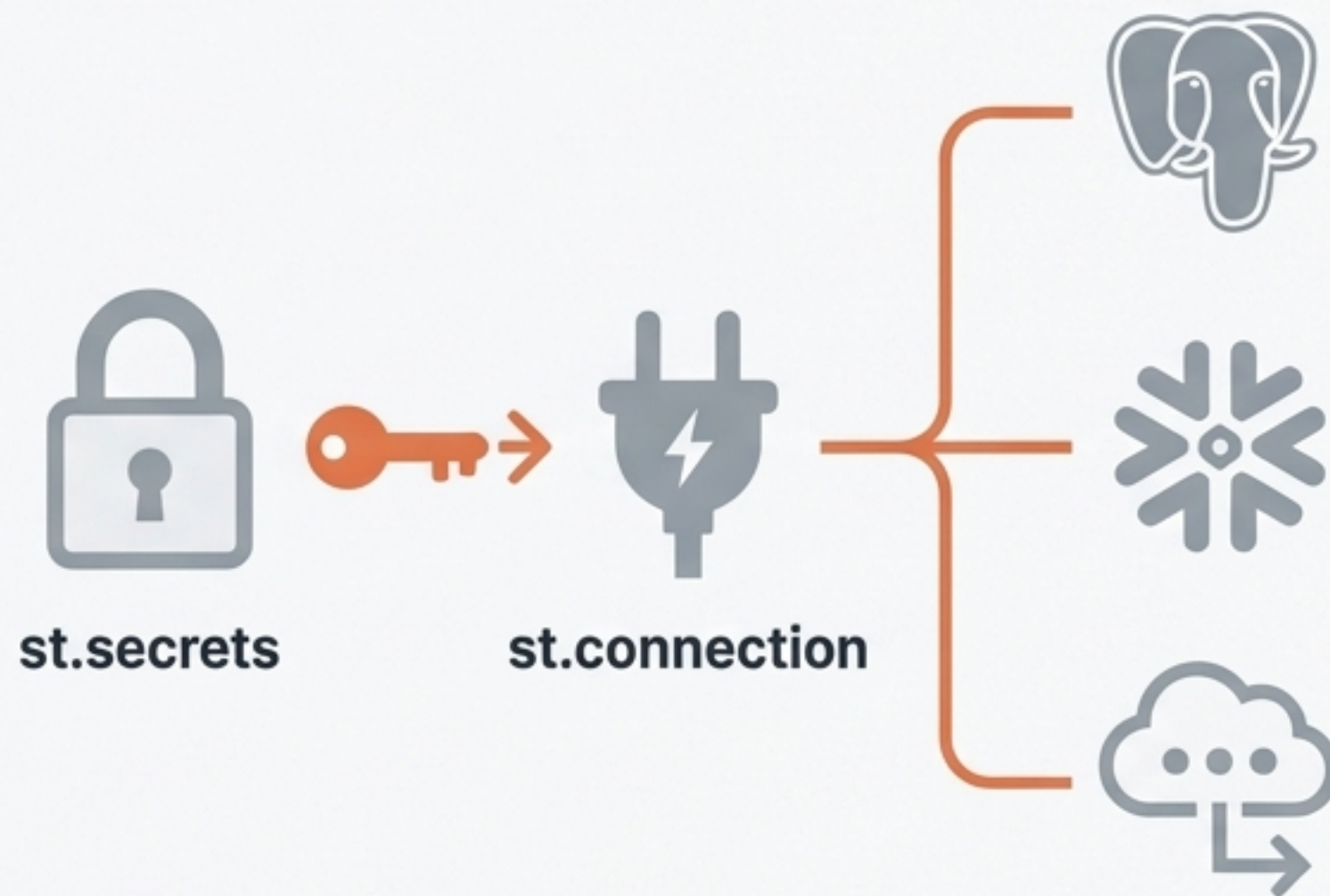
프로덕션 환경으로: 안전한 데이터 연결과 보안

Standardized Data Access: ``st.connection``

- 데이터 소스(SQL, Snowflake 등)에 대한 연결 객체를 생성하고 관리하는 통합 팩토리입니다.
- 내부적으로 `@st.cache_resource`를 활용하여 연결을 효율적으로 재사용합니다.
- 개발자는 연결 로직이 아닌 비즈니스 로직에 집중할 수 있습니다.

Secure Credential Management: ``st.secrets``

- API 키, DB 비밀번호 등 민감 정보는 코드와 분리하여 ``.streamlit/secrets.toml`` 파일에 저장합니다.
- 앱 코드에서는 ``st.secrets["db_password"]``와 같이 딕셔너리 형태로 안전하게 값에 접근합니다.



These features make Streamlit ready for serious, enterprise-level deployment.

Streamlit의 진화: 스크립팅 툴에서 완전한 앱 프레임워크로



과거 (Past)

간단한 데이터 시각화 및
스크립팅 도구



현재 (Present)

``st.session_state``, ``st.form``,
``st.fragment`` 등의 도입으로
복잡한 상태 관리와 부분 렌더링
렌더링이 가능한 **완전한 웹
애플리케이션 프레임워크**로 성장



미래 (Future)

``st.chat_message``,
``st.write_stream``과 같은 LLM
특화 기능으로 **AI 에이전트 및 챗
봇 인터페이스 프로토타이핑의
표준 도구**로 자리매김

성공을 위한 블루프린트: 핵심 원칙

1

재실행 모델을 수용하라
(Embrace the Rerun Model)



콜백이 아닌 데이터 흐름의
관점에서 생각하십시오.

2

상태와 캐싱을 마스터하라
(Master State & Caching)



``st.session_state``와
``@st.cache_*``는 복잡성과
성능의 열쇠입니다.

3

적재적소에 컴포넌트를 사용하라
(Use the Right Component)



레이아웃, 컨테이너부터 채팅
요소까지 API의 모든 기능을
활용하십시오.

Streamlit의 직관적인 API 이면에는 강력한 아키텍처가 있습니다.
이 아키텍처를 이해하는 것이 Streamlit의 모든 잠재력을 끌어내는 열쇠입니다.